

COMPUTER VISION

ASSIGNMENT 2

Implementation of Canny's Edge Detector

Edge detection is an important step for many downstream tasks. In this assignment, we are going to implement a very important edge detector called Canny's edge detector.

Guidelines

- 1) Submit all of your code and results to GitHub incrementally. By incremental, we mean you should show your day-to-day progress, pushing updates as you work. Uploading all of your code at once will be flagged and will affect the grade.
- 2) In the classroom you have to submit only a .pdf file with the name **RollNumber_FirstName_02.pdf**. Please note: Do not submit code or results on Google Classroom.
- 3) **Contents of the GitHub Repository**
 - a) All of your work must be properly organized into classes and modules. You should have a single **main.py** file that imports and runs all the required classes and functions, as explained in the TA session. Please do not write all of your code in one file.
 - b) **Code Submission**

Python file that could be opened in VS-Code and could be run from the terminal. WE WILL NOT ACCEPT jupyter or other notebooks.

On top of the file, the first three lines should be commented lines containing Your Roll Number, Your Name, Assignment Number respectively. In your main.py, define proper input arguments so that the script can be executed with the following command. It should take an input folder (to read images), and save the processed results to the specified output folder with specified image extension

```
python3 folder_name/main.py --input_folder "path_to_input_folder" \
--output_folder "path_to_output_folder" --input_ext "png" --output_ext "png"
```
 - c) **Results**

A folder with result images for all sample images given. NO SCREENSHOTS, properly save the images.
 - d) **Readme.md**

Should explain how to run your code in the file.
 - e) **Report.pdf**

Explanation of your implementation and highlighting interesting results. Properly save images and do not take the screenshots of the results and add them in the report. (Also submit it on google classroom)
 - f) **mathematicalPart.pdf**
- 4) Email instructor or TA if there are any questions. You can discuss with each other but cannot look at or use anyone's code or implementation. Do not share your code even after the submission.
DO NOT GOOGLE SEARCH OR use LLM's (ChatGPT or similar), DO NOT SEEK HELP FROM CODE COMPLETION OR IMPLEMENTATION HELPING TOOLS (e.g. co-pilot). See if you can implement something when no such help is there
- 5) This document has two parts, first **Implementation** and second is the **mathematical** part. The Implementation part has two portions, the first **Steps for implementation** and second **Problems**.
- 6) Before solving the problem section, first, go through the steps which will help you implement Canny's edge detector step by step.
- 7) You cannot use any built-in function unless specifically mentioned.
- 8) **Note: 10% deduction on obtained marks for each late day.**

Implementation

There are some basic steps for the implementation of a Canny edge detector.

1. Generation of Masks
2. Applying Masks to Images
3. Compute gradient magnitude
4. Compute gradient Direction
5. Non-Maxima Suppression
6. Hysteresis Thresholding

Steps for implementation:

1. **Generation of Masks:** This module requires the value of **sigma** as an input and generates x- and y-derivative masks as output.
 - Mask size: To generate the masks, the first step is a reasonable computation of the mask size. Mask size depends on the value of sigma and T. Formula of size of half mask is as follows:

$$sHalf = \text{round}(\sqrt{-\log(T) * 2 * \sigma^2})$$

- Where sigma and T are input parameters.
 - i. Sigma can be **0.5, 1, 1.5...** The lower limit of sigma is 0.5.
 - ii. Width of the mask is based on parameter T. T can be any value **between 0 and 1**. For example **T=0.3**. The above formula will give us the size of half mask.
- The total mask size would then be computed as follows:

$$N = 2 * sHalf + 1 \text{ (total mask size)}$$

Example:

`sigma=0.5;      T=0.3`

`sHalf = round(sqrt(-log(T) * 2 * sigma^2))      =>    1`

`N= 2*sHalf +1      =>    3`

`[Y, X] = np.meshgrid ( (-sHalf : sHalf+1), (-sHalf : sHalf+1) )    => Y= [[-1,0,1] [-1,0,1] [-1,0,1]]`

- We will be using the **1st derivative of Gaussian** in x- and y-directions. The Gaussian formula is as follows:

$$G(x, y) = \exp(-(x^2 + y^2)/(2 * \sigma^2))$$

- Now take the first derivative of Gaussian w.r.t 'x' and call it 'fx' and put the value of X, Y in it. Repeat this procedure w.r.t 'y' as well. These (Gx, Gy) are the two masks that will be convolved with the image in the next step.
- Multiply the Gx and Gy with 255 to scale up and then round off these values so that convolution is performed with integer values rather than floats.

Save the scale factor, because gradient magnitude will be later scaled down (after convolution) by the same factor.

Note:

- **You should write a function to calculate the gradient of gaussian not just save it:**
 - `calculate_gradient (filter_size, sigma)`
 - **Write a separate function to calculate filter size named**
 - `calculate_filter_size (sigma, T)`
2. **Applying Masks to Images:** The masks are applied to the images using convolution. Store convolution results in `fx` and `fy`.

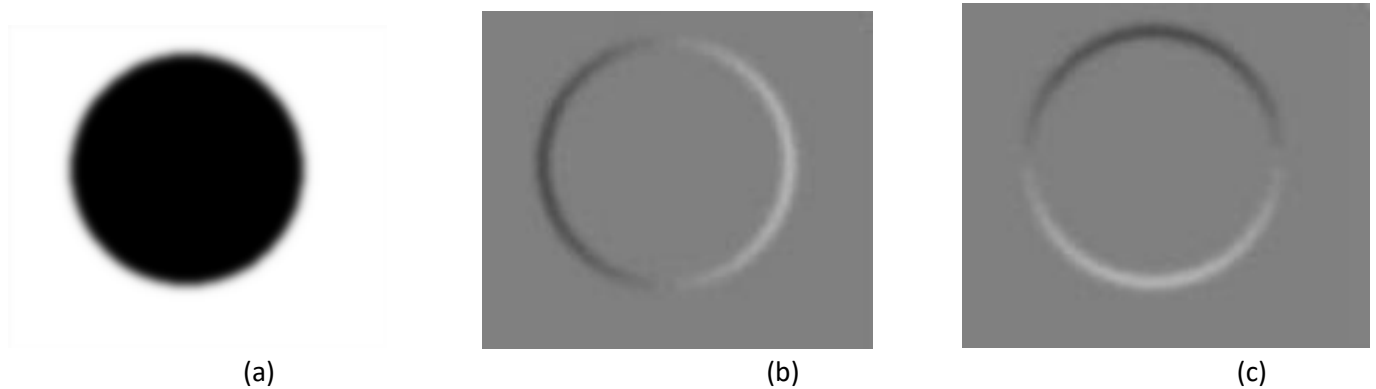


Figure 1: Image (a) is an input image 'circleBlured.png'. Image (b) is generated after convolving the input image with G_y (Gaussian derivative w.r.t 'y'). Image (c) is generated after convolving the input image with G_x (Gaussian derivative w.r.t 'x').

3. **Compute gradient magnitude:** Compute the gradient magnitude of images (`fx`, `fy`) at each pixel after convolving with masks (G_x , G_y). M is computed from `fx` and `fy` images, using the magnitude formula:

$$M = \sqrt{f_x^2 + f_y^2}$$



Figure 2: Gradient magnitude of input image normalized b/w 0-255

Note: The result is then scaled down by the same factor which was used to scale up the masks. To write output to image files, the min and max values are scaled to 0 and 255 respectively.

4. **Compute gradient Direction:** Compute the gradient direction of images (fx, fy) at each pixel after convolving with masks (Gx, Gy). Phi is computed using atan2 function by the following a formula:

$$\theta = \arctan \frac{f_y}{f_x}$$

Note: Convert the angle returned by math.atan2 function to degrees and add 180 to get an output range of 0-360 degrees.

5. **Non-Maxima Suppression:** Non-Maxima suppression step makes all edges in M one pixel thick.

- The first step is to quantize the gradient direction into just four directions.

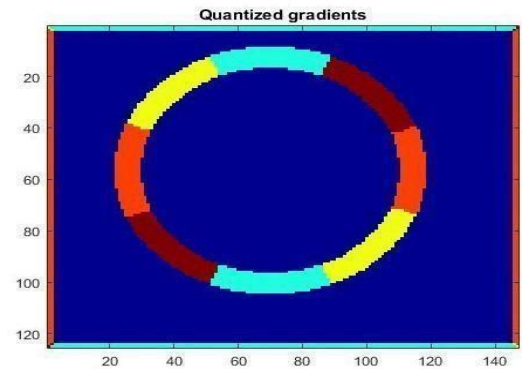


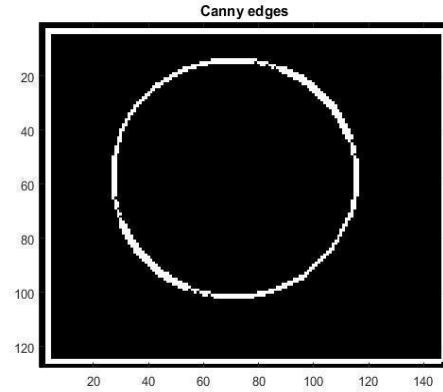
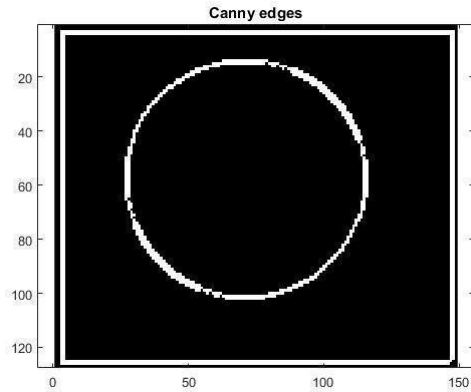
Figure 3: Image after quantizing gradient directions

Value assigned	Angle
0	0 to 22.5 157.5 to 202.5 337.5 to 360
1	22.5 to 67.5 202.5 to 247.5
2	67.5 to 112.5 247.5 to 292.5
3	112.5 to 157.5 292.5 to 337.5

Table 1 Quantize gradient directions

- The next step is to pick two neighbors of each edge point **along the gradient direction**. This is because the gradient direction is perpendicular to the edge, and therefore, this is the direction in which we are searching for edge points.
 - Therefore, the two neighbors that need to be picked for comparison are the north and south neighbors. If the edge point (r, c) is greater than both these neighbors, then it is maintained in M otherwise it is made zero.
6. **Hysteresis Thresholding:** The final step in Canny's edge detection algorithm is to apply two thresholds to follow edges.
 - First make the border pixels zero, so that finding neighbors does not go out of bounds of the image.
 - Next, the image is scanned from left to right, top to bottom. The first pixel in non-maxima suppressed magnitude image which is above a certain threshold, **Th**, is declared an edge.

- Then all its neighbors are recursively followed, and that above threshold, T_l , is marked as an edge.
- Thus, there are really two stopping conditions:
 - i. if a neighbor is below T_l , we won't recurs on it.
 - ii. if a neighbor has already been visited, then we won't recurs on it.



Problems:

For Assignment 1, you have to implement the Canny Edge Detector. Test Images are given in the assignment folder. Create separate functions for all 4 steps. Failing to do so will result in the deduction of marks. These results should be generated from your implementation.

For each image, you have to submit (no screenshots, properly save images)

- a) images showing gradients in X and Y direction (10)
- b) image showing the Magnitude of gradients at each pixel (10)
- c) Image of quantized orientations (15)
- d) Final Result for sigma = 1, sigma = 0.5 and sigma = 2 (20)
- e) For sigma = 1, submit 2 different results for 2 different pairs of T_h , T_l in Hysteresis thresholds. (25)

Note: Perform each experiment for all images provided and save resultant images in the Results folder and name them as

- [original Name] _fx_[sigmavalue].jpg
- [original Name] _fy_[sigmavalue].jpg
- [original Name]_magnitude_[sigmavalue].jpg
- [original Name] _ quantized_[sigma value].jpg

Mathematical Part

**Show all the steps, clearly indicate what you are doing and why you are doing it.
Specify any assumptions you have made.**

Problem 1: Calculate and plot (using matplotlib.pyplot's functions e.g. plot_surface) derivative and double derivative of Gaussian filter. **(10)**

Problem 2: Prove that Convolutions are commutative $a*b=b*a$ ($*$ => convolution) **(05)**

Problem 3: Calculate the rank of given matrices. **(05)**

$$A = \begin{bmatrix} 1 & 2 & 3 \\ 3 & 6 & 9 \end{bmatrix}$$

$$B = \begin{bmatrix} 1 & 2 & 3 \\ 4 & 5 & 6 \\ 7 & 8 & 9 \end{bmatrix}$$

Note: add page number on top of every page, add your name on the top Right of every page. Scan the pages into one pdf and make submission along with the code.